Unit 2- Classification

Fully Connected Networks Applied to Multiclass Classification:

1. **build a network** that can take an image of a **handwritten digit as input**,
2. **identify** which one of the ten digits 0 through 9 the image represents
3. **present** this information on **its outputs.**

Before showing how to build such a network, some concepts that are central to both traditional machine learning (ML) and deep learning (DL),

1. **namely, datasets and**
2. **generalization.**

Introduction to Datasets Used When Training Networks

We train a neural network by presenting an input example to the network.
We then compare the **network output** to the**expected output** and **use gradient descent** to adjust **the weights**to try to makethe network provide the correct output for a given input.

A reasonable question isfrom where to get these training examples that are needed to train the network?

In real applications of DL, obtaining these training examples can be a big challenge.

- One of the key reasons that DL has gained so much attraction lately is that **large online databases of images, videos, and natural language text** have made it possible to **obtain large sets of training data.**
- If a supervised learning technique is used, **it is not sufficient to obtain the input** to the network.
- We also need to know the **expected output, the ground truth, for each example**. The process of associating each training input with an expected output is known as **labeling.**
- That is, **a human must add a label to each example**, detailing whether it is a dog, a cat, or a car.
- This **process can be tedious** because we often need many thousands of examples to achieve goodresults.

- Starting to experiment with **DL might be hard** if the first step involved **putting together a large collection of labeled training examples**.
- Fortunately, **other people have already done** so and have made these examples publicly available.
- This is where the **concept of datasets comes** in.
- A (labeled) dataset consists of a collection of labeled training examples that can be used for training ML models.

Classic Iris Dataset (Fisher, 1936)

- The classic Iris Dataset (Fisher, 1936), which is likely the first widely available dataset.
- It contains **150 instances of iris flowers**, each instance belonging to **one of three** iris species.
- Each instance consists of **four measurements (sepal length and width, petal length and width)** of the particular plant.

MNIST dataset.

- The Modified National Institute of Standards and Technology (MNIST) database of handwritten digits, also known simply as the **MNIST dataset.**
- The MNIST dataset contains **60,000 training images** and **10,000 test images.**
- The dataset **consists of labels** that describe which digit each image represents.
- The **original images are 32×32 pixels,** and the **outermost two pixels** around **each image are blank**, so the actual image content is found in the centered 28×28 pixels.
- The **blank pixels have been stripped out**, so each image is **28×28 pixels.**
- Eachpixel is represented by a **grayscale** value ranging from **0 to 255.**
- The source ofthe handwritten digits is a mix of **employees at the American Census Bureau and American high school students.**
- The dataset was made available in **1998.**

Exploring the Dataset

Code Snippet 4-1:Load the MNIST Dataset and Inspect Its Dimensions

```
import keras
from keras.models import Sequential
fromkeras.layers import Dense,Flatten
```

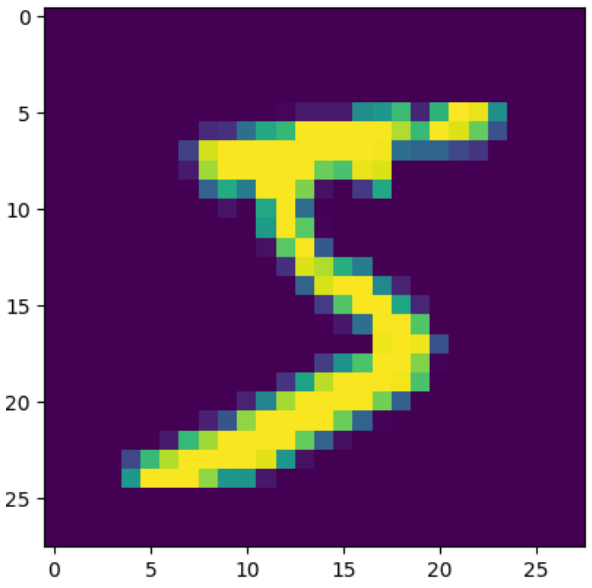
```
(X_train,y_train),(X_test,y_test) = keras.datasets.mnist.load_data()
```

```
X_train.shape
```
(60000, 28, 28)

```
y_train.shape
```
(60000,)

Code Snippet 4-2Print Out One Training Example

```
import matplotlib.pyplot as plt
plt.imshow(X_train[0])
```



Human Bias in Datasets

- Because ML models learn from input data, they are susceptible to the **garbage-in/garbage-out (GIGO) problem.**
- It is therefore important to **ensure that any used dataset is of high quality.**
- A subtle problem to look out for is if the **dataset suffers from human bias** (or any other kind of bias).

(CelebA) dataset

- A popular dataset available online is the CelebFaces Attributes (CelebA) dataset which is derived from the CelebFaces dataset.
- It consists of a large number of images of **celebrities' faces.**
- It has a **disproportionate number of young, white individuals**.
- Because of this imbalance, ML models trained on this dataset may not perform well for **older or dark-skinned individuals.**

MNISTdataset

- The handwritten digits in **MNIST originate from the American Census Bureau employees and American high school students**
- The digits will therefore be biased toward how people in the United States write digits.
- In particular, in some European and Latin American countries, it is common to add **a second horizontal line when writing the digit 7.**
- If you explore the MNIST dataset, only two of the 16 examples in above figure **have a second horizontal line**
- The dataset is biased toward how people in the United States write these digits.
- Therefore, it may well be that a **model trained on MNIST works better for people in the United States** than for people from countries that use a different style for the digit 7.

Self driving Cars

- Consider a self-driving car where the model needs to distinguish between a human being and a lessvulnerable object.
- If the model has not been trained on a diverse dataset with enough representation of minority groups, then it can have fatal consequences.
  - A good dataset does not always have **to mirror real-world** distributions exactly.
  - In safety-critical applications like self-driving cars, datasets should include **overrepresented examples of rare but dangerous events**—such as an airplane making an emergency landing on a road—to ensure the model can handle them.

- Therefore, a good dataset might **well contain an overrepresentation of such events** compared to what is present in the real world.

Gebru and colleagues (2018) proposed datasheets for datasets to **address this problem.** Each released dataset should be accompanied by a datasheet that describes itsrecommended use and other details.

Training Set, Test Set, and Generalization

- The goal for an ML model is not just to provide correct predictions for data that it has been **trained on**; the more important goal is to provide correct predictions for previously **unseen data**.
- Therefore, we typically divide our dataset into a training dataset and a test dataset.
- **The training dataset** is used to **train** the model, and **the test dataset** is used to later **evaluate how** well the model was able to generalize to previously unseen data.
- If it turns out that the model does well on the training dataset but does poorly on the test dataset, then that is an indication that the model **has failed to learn the general solution**

For example, it might have memorized only the specific training examples. To make this more concrete, consider the case of teaching children addition.

1 + 1 = 2
2 + 2 = 4
3 + 2 = 5

It successfully repeat the answer when asked, What is 3 + 2?
yet be unable to answer, What is 1 + 3? or even What is 2 + 3?

It indicate that the child **has memorized** the three examples **but not understood the concept of addition.**

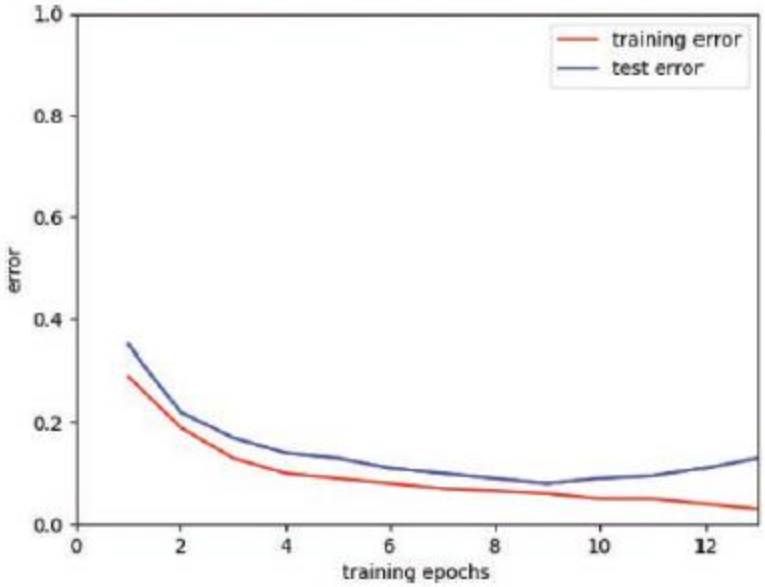


Figure 4-2 How training error and test error can evolve during learning process

- The training error will show **a downward trend** until it finally flattens out.
- The test error, on the other hand, will often show a **U-curve** where it decreases in the beginning but then at some point starts increasing again.

Reason for Poor Generalization:

Overfitting

- If test error starts **increasing** while the training error is still **decreasing**, then that is a sign that the model is **overfitting to the training data.**
- That is, it learns a function that does really well on the training data but that is not useful on not-yet-seen data.
- **Memorizing individual examples** from the training set is one strong form of **overfitting**

Another reason for Poor Generalization

- Overfitting is not the only reason for lack of generalization. It can also be that the training examples are **simply not representative of the examples in the test set**

How to Reduce Overfitting?

- **Increasing the training dataset size** can help.
- **Regularization techniques** (various methods to prevent overfitting).

- **Early stopping**: Monitor test error during training and stop when it starts increasing.
- **Checkpointing:** Save different versions of the model at intervals and later select the one with the lowest test error.

Hyperpar meter Tuning and Test Set Information Leakage

Information Leakage

- If information from **the test set** is used in any way during training, the model may end up memorizing the test set.
- This creates a **false impression of the model's accuracy**, making it seem better than it actually is in real-world use.

Hyperparameter

- When training a model, there is sometimes a need to tune various parameters that are **not adjusted by the learning algorithm** itself.

Examples include:

1. **Learning rate** (how fast the model updates weights)
2. **Network topology** (number of neurons, layers, and their connections)
3. **Activation function** (the function used to introduce non-linearity in neurons)

- If we **change these hyperparameters** on the basis of how the model performs on **the test set**
- Thenthe test set risks influencing the **training process.**
- This causes **information leakage** from the test set to the training process.

To avoid leakage during Hyperparameter tuning

- One way to avoid such leakage is to introduce an intermediate **validation dataset.**
- It is used for evaluating hyperparameter settings before doing a final evaluation on the test dataset.

Training vs. Inference:

Training: The process where the network learns by adjusting its weights based on data.

Inference: The process of using the network without adjusting the weights is known as inference because the network is used to infer a result

- It is often the case that the training process is done only before the network is deployed in a production setting
- And once the network is deployed, it is used onlyfor inference.

Different Hardware for Training and Inference

In such cases, training and inference may well be done **on different hardware implementations.**

- For instance, **training** might be done on **servers in the cloud**, and
- **Inference**might be done on a less powerful device such as a **phone or tablet.**

Extending the Network and Learning Algorithm to do Multiclass classification

- One naïve way of doing is to simply create **ten different networks.** Each of them is responsible for identifying one specific digit type.
  This is a **inefficient approach.**
- There are some commonalities among the different digits, so it is more efficient if each "digit identifier" shares many of the neurons.
- This strategy also forces the **shared neurons** to generalize better and can reduce the risk of overfitting. (**single network** with shared neurons is better).

sparse encoding vs dense encoding

- For arranging a network to do multiclass classification, create **one output neuron per class** and teach the network to output a one-hot encoded number.
- One-hot encoding implies that only one of the outputs is excited at any one point in time.
- One-hot encoding is an example of a **sparse encoding,** which means that most of the signals are 0.
- Binary encoding reduces the number of output neurons, but it is not most suitable encoding for a neural network.

- Binary encoding is an example of a **dense encoding**, which means that we havea good mix of 1s and 0s.

Network for Digit Classification:

(write the explanations of python code for MNIST dataset)

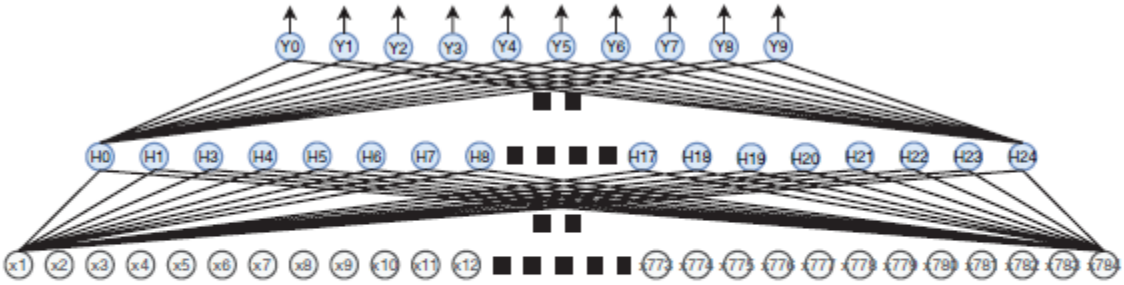


Figure 4-3 Network for digit classification. A large number of neurons and connections have been omitted from the figure to make it less cluttered. In reality, each neuron in a layer is connected to all the neurons in the next layer.

Programming Example: Classifying Handwritten Digits

Refer the code in classroom

Loss Function for Multiclass Classification

- XOR problem, we used mean squared error (MSE) as our loss function
- For image classification problem, defining our loss (error) function as the sum of the squared error for each individual output

$$Error = \frac{1}{m}\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}(y_j^{(i)} - \hat{y}_j^{(i)})^2$$

where m is the number of training examples and
n is the number of outputs.

For a single training example, we end up with the following

$$Error = \sum_{j=0}^{n-1}(y_j - \hat{y}_j)^2$$

where n is the number of outputs and $\hat{y}_j$ refers to the output value of neuron Y_j.

To simplify our derivative later, **divide by 2** because minimizing a loss function that **is scaled by 0.5** will result in the same optimization process as minimizing the unscaled loss function

$$Error : e(\hat{\boldsymbol{y}}) = \sum_{j=0}^{n-1}\frac{\left(y_j - \hat{y}_j\right)^2}{2}$$

The error functions as a function of **ˆy**, which represents the output of the network. Note that **ˆy** is now a vector because the assumed network has multiple output neurons

Given the loss function, compute the **error term for each of the n output neurons**. The following formula shows the error term for neuron Y_1 with output value $\hat{y}_1$

$$\frac{\partial e}{\partial \hat{y}_1} = \sum_{j=0}^{n-1}\frac{\partial\frac{\left(y_j - \hat{y}_j\right)^2}{2}}{\partial \hat{y}_1} = \frac{2(y_1 - \hat{y}_1)}{2}\cdot(-1) = -\left(y_1 - \hat{y}_1\right)$$

- When computing the derivative of the loss function with respect to a specific output, all the other terms in the sum are constants (derivative is 0),

- That is, the error term for neuron Y_2 is $-(y_2 - \hat{y}_2)$
- The general case, the error term for Y_j is $-(y_j - \hat{y}_j)$

Mini-Batch Gradient Descent

- **Types of Gradient Descent**
  - **Stochastic Gradient Descent (SGD):** Updates weights after each individual training example (fast but noisy).
  - **Batch Gradient Descent:** Computes the gradient over the entire dataset before updating weights (accurate but slow).
  - **Mini-Batch Gradient Descent:** Uses a small subset (mini-batch) of training examples, balancing accuracy and efficiency.
- **Advantages of Mini-Batch Gradient Descent**
  - More frequent updates than batch gradient descent.
  - More accurate gradient estimation than SGD.
  - Efficient on modern hardware, especially **GPUs**.
- **Terminology Confusion**
  - **Batch Gradient Descent** → Uses the full dataset.
  - **SGD (Stochastic Gradient Descent)** → Uses **one** training example (sometimes called **online learning**).
  - **Mini-Batch Gradient Descent** → Uses a small batch **Mini-batch size (batch size)** is a tunable hyperparameter (common range: **32–256**).
- **Implementation Efficiency**
  - **Mini-batches are represented as matrices**, enabling efficient computation.
  - **Matrix-matrix multiplication** speeds up training on optimized platforms (e.g., TensorFlow).